

# Actividad - Completar Task Manager

React + TypeScript | Estado, props y componentes

## Objetivo

Completar el proyecto **Task Manager** iniciado en clase, usando React + TypeScript, componentes, props y estado.

En clase ya se avanzó hasta:

- Proyecto funcionando en React.
- Archivos convertidos a .tsx.
- Uso de TypeScript en props.
- Estado de tareas en App.
- Componente TaskInput para agregar tareas.

Ahora deben completar las funcionalidades restantes.

## 1. Funcionalidades obligatorias

- Agregar tareas.
- Eliminar tareas.
- Marcar tareas como completadas.
- Mostrar un mensaje cuando no haya tareas.
- Mostrar un contador de tareas.

## 2. Componentes que deben existir

El proyecto debe tener estos componentes dentro de **src/components**:

- Header.tsx
- TaskInput.tsx
- TaskList.tsx
- TaskCard.tsx
- EmptyState.tsx
- Footer.tsx

Ya tienen algunos creados. Deben completar o crear los que falten.

## 3. Estructura esperada del proyecto

```
src
|-- components
|   |-- Header.tsx
|   |-- TaskInput.tsx
|   |-- TaskList.tsx
|   |-- TaskCard.tsx
|   |-- EmptyState.tsx
```

```
|  `-- Footer.tsx
|-- App.tsx
|-- App.css
|-- index.css
|-- main.tsx
`-- vite-env.d.ts
```

## 4. Tipo de dato obligatorio

En **App.tsx** deben manejar las tareas con este tipo:

```
type Task = {
  id: number;
  text: string;
  completed: boolean;
};
```

Cada tarea debe tener:

- id: identificador único.
- text: texto de la tarea.
- completed: indica si está completada o no.

## 5. App.tsx debe manejar el estado

El estado principal debe estar en **App.tsx**.

```
const [tasks, setTasks] = useState<Task[]>([
  { id: 1, text: "Estudiar React", completed: false },
  { id: 2, text: "Practicar TypeScript", completed: false },
  { id: 3, text: "Entender estado", completed: true }
]);
```

**App.tsx** debe contener las funciones para:

- Agregar tarea.
- Eliminar tarea.
- Marcar tarea como completada o pendiente.

## 6. Función para agregar tarea

```
const addTask = (text: string) => {
  const newTask: Task = {
    id: Date.now(),
    text: text,
    completed: false
  };

  setTasks([...tasks, newTask]);
};
```

## 7. Función para eliminar tarea

Deben crear una función que reciba el **id** de la tarea y la elimine:

```
const deleteTask = (id: number) => {
  const updatedTasks = tasks.filter((task) => task.id !== id);
  setTasks(updatedTasks);
};
```

## 8. Función para marcar tarea como completada

Deben crear una función que cambie el valor de **completed**:

```
const toggleTask = (id: number) => {
  const updatedTasks = tasks.map((task) => {
    if (task.id === id) {
      return {
        ...task,
        completed: !task.completed
      };
    }

    return task;
  });

  setTasks(updatedTasks);
};
```

## 9. TaskCard.tsx

**TaskCard** debe mostrar una tarea individual. Debe recibir:

- task
- onDeleteTask
- onToggleTask

```
type Task = {
  id: number;
  text: string;
  completed: boolean;
};

type TaskCardProps = {
  task: Task;
  onDeleteTask: (id: number) => void;
  onToggleTask: (id: number) => void;
};

function TaskCard(props: TaskCardProps) {
  return (
    <li className={props.task.completed ? "task completed" : "task"}>
      <input
        type="checkbox"
        checked={props.task.completed}
        onChange={() => props.onToggleTask(props.task.id)}
      />

      <span>{props.task.text}</span>

      <button onClick={() => props.onDeleteTask(props.task.id)}>
        Eliminar
      </button>
    </li>
  );
}

export default TaskCard;
```

## 10. TaskList.tsx

**TaskList** debe recibir la lista de tareas y mostrar un **TaskCard** por cada tarea. También debe mostrar **EmptyState** si no hay tareas.

```
import TaskCard from "../TaskCard";
import EmptyState from "../EmptyState";

type Task = {
  id: number;
  text: string;
  completed: boolean;
};

type TaskListProps = {
  tasks: Task[];
  onDeleteTask: (id: number) => void;
  onToggleTask: (id: number) => void;
};

function TaskList(props: TaskListProps) {
  if (props.tasks.length === 0) {
    return <EmptyState />;
  }

  return (
```

```

    <ul>
      {props.tasks.map((task) => (
        <TaskCard
          key={task.id}
          task={task}
          onDeleteTask={props.onDeleteTask}
          onToggleTask={props.onToggleTask}
        />
      )
    )}
  </ul>
);
}

export default TaskList;

```

## 11. EmptyState.tsx

Crear un componente llamado **EmptyState.tsx**. Debe mostrar un mensaje cuando no haya tareas:

```
function EmptyState() {
  return (
    <p className="empty-state">
      No hay tareas todavía. Agrega una nueva tarea.
    </p>
  );
}

export default EmptyState;
```

## 12. Footer.tsx

Crear un componente llamado **Footer.tsx**. Debe mostrar total, completadas y pendientes:

```
type FooterProps = {
  total: number;
  completed: number;
  pending: number;
};

function Footer(props: FooterProps) {
  return (
    <footer>
      <p>Total de tareas: {props.total}</p>
      <p>Completadas: {props.completed}</p>
      <p>Pendientes: {props.pending}</p>
    </footer>
  );
}

export default Footer;
```

## 13. App.tsx final esperado

**App.tsx** debe unir todos los componentes:

```
import { useState } from "react";
import "./App.css";
import Header from "./components/Header";
import TaskInput from "./components/TaskInput";
import TaskList from "./components/TaskList";
import Footer from "./components/Footer";

type Task = {
  id: number;
  text: string;
  completed: boolean;
};

function App() {
  const [tasks, setTasks] = useState<Task[]>([
    { id: 1, text: "Estudiar React", completed: false },
    { id: 2, text: "Practicar TypeScript", completed: false },
    { id: 3, text: "Entender estado", completed: true }
  ]);

  const addTask = (text: string) => {
    const newTask: Task = {
      id: Date.now(),
      text: text,
      completed: false
    };
  };
}
```

```

    setTasks([...tasks, newTask]);
  };

  const deleteTask = (id: number) => {
    const updatedTasks = tasks.filter((task) => task.id !== id);
    setTasks(updatedTasks);
  };

  const toggleTask = (id: number) => {
    const updatedTasks = tasks.map((task) => {
      if (task.id === id) {
        return {
          ...task,
          completed: !task.completed
        };
      }
      return task;
    });
    setTasks(updatedTasks);
  };

  const completedTasks = tasks.filter((task) => task.completed).length;
  const pendingTasks = tasks.length - completedTasks;

  return (
    <div className="app-container">
      <Header />
      <TaskInput onAddTask={addTask} />
      <TaskList
        tasks={tasks}
        onDeleteTask={deleteTask}
        onToggleTask={toggleTask}
      />
      <Footer
        total={tasks.length}
        completed={completedTasks}
        pending={pendingTasks}
      />
    </div>
  );
}

export default App;

```

## 14. Estilos obligatorios

Deben mejorar la apariencia usando CSS. El proyecto debe tener:

- src/index.css
- src/App.css

Opcionalmente pueden crear archivos por componente:

- Header.css
- TaskCard.css
- TaskInput.css

La aplicación debe verse ordenada y cumplir:

- Aplicación centrada.
- Contenedor tipo tarjeta.
- Input limpio.
- Botones visibles.
- Espaciado consistente.
- Tareas completadas visualmente diferentes.

Ejemplo para tarea completada:

```
.completed span {  
  text-decoration: line-through;  
  color: gray;  
}
```

## 15. Entrega

Para la próxima clase deben mostrar:

- Proyecto corriendo con npm run dev.
- Código en VS Code.
- Aplicación en navegador.
- Agregar tarea funcionando.
- Eliminar tarea funcionando.
- Marcar completada funcionando.
- Contador funcionando.
- Código subido a GitHub.

## 16. Criterios de revisión

Se revisará:

- Uso correcto de componentes.
- Uso correcto de TypeScript.
- Uso correcto de props.



- Estado manejado desde App.
- Funcionalidad completa.
- Organización del código.
- Estilos básicos aplicados.
- Proyecto subido a GitHub.

## **Mensaje final para estudiantes**

Lo que no alcancemos a terminar en clase queda como tarea. La meta es que para la próxima sesión tengan el Task Manager funcional, porque después lo vamos a preparar para conectarlo con backend o API.